Jsp - Servlet

Chapitre 1: Premiers pas avec les Servlets et les JSP

Dr Ahmad Khoureich Ka

UADB

L3 D2AW

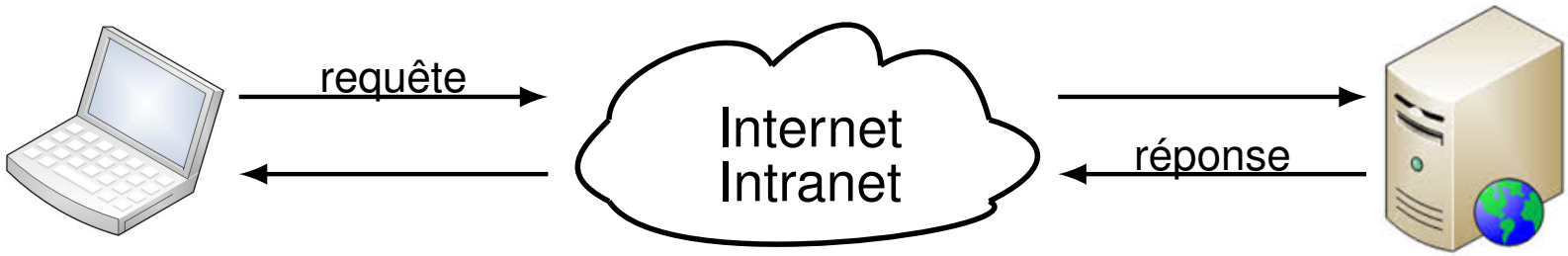
1. Application web
   - C'est quoi une application web ?
2. Servlet
   - Fonctionnement
   - Cycle de vie
   - Structure de base
   - Un exemple de servlet
   - Fichiers de configuration
3. Java Server Page
   - Les éléments de scripts
   - Les directives
   - Objets implicites

Application web ?

- Une application web est une application client-serveur.
- Elle est exécutée par un logiciel serveur (le serveur).
- L'accès à l'application web se fait depuis un logiciel client par exemple un navigateur web.
- La communication entre le client et le serveur se fait selon le protocole HTTP à travers un réseau (internet ou intranet).
- Le serveur peut communiquer en même temps avec plusieurs clients.

Application web

- Le client (généralement un navigateur web)
  - envoie une requête
  - reçoit et présente la réponse
- Le serveur HTTP (logiciel)
  - reçoit les requêtes
  - produit et envoie les réponses



Les Servlets

Le servlet : Qu'est-ce que c'est

- Le servlet assure le coté dynamique d'une application web Java.
- Un servlet est une classe Java qui implémente des méthodes de traitement de requêtes et éventuellement de génération de réponses.
- Une fois compilé et chargé, le servlet est associé à une url et s'exécute coté serveur dans un conteneur de servlets.

Le servlet : Fonctionnement

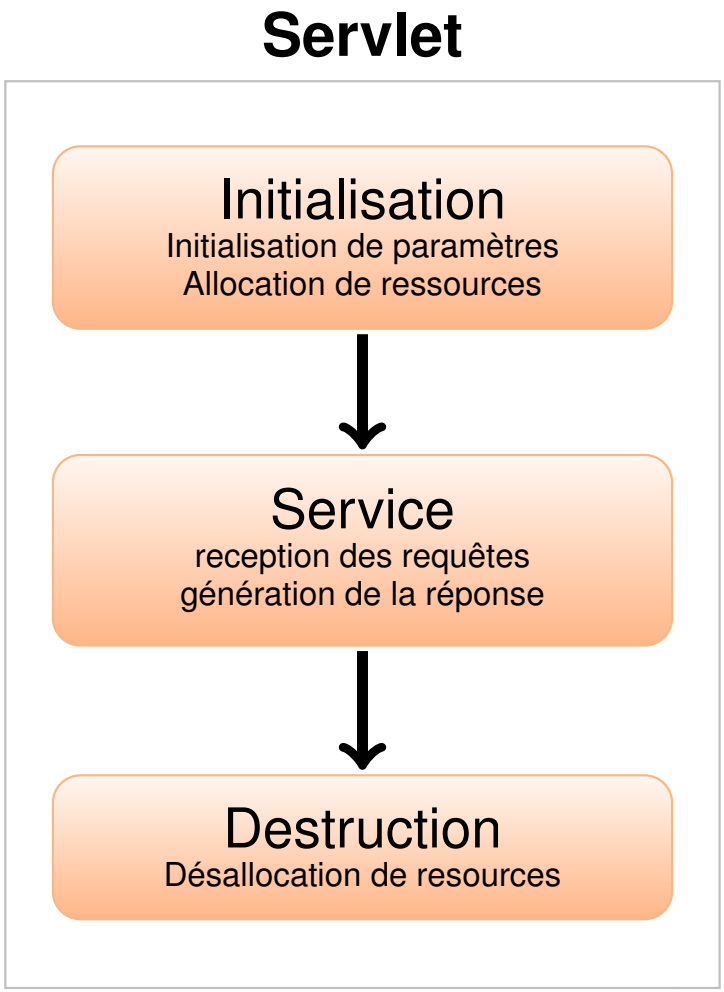
Lorsque le serveur reçoit une requête, il charge (si ce n'est pas encore fait) le servlet qui doit la traiter.

Le servlet effectue les actions suivantes :

1. lire la requête envoyée par le client (données de formulaires et en-têtes HTTP de requête) ;
2. générer la réponse (codes de statut, en-têtes HTTP de réponse, texte, html, images, etc.) ;
3. envoyer la réponse vers le client.

Le servlet : Cycle de vie

- Un servlet étend la classe abstraite **`javax.servlet.http.HttpServlet`**



Cycle de vie : Phase Initialisation

- 2 versions de la méthodes `init()` héritées de `javax.servlet.GenericServlet`
- Exécuter une seule fois lors du chargement du servlet.

- Version 1

```
public void init(ServletConfig config) throws ServletException {
  super.init(config);
  ...
}
```

- Version 2

```
public void init() throws ServletException {
  ...
}
```

Cycle de vie : Phase Initialisation

Remarques :

- L'objet `ServletConfig` peut être obtenu avec la méthode `getServletConfig()` de `javax.servlet.GenericServlet`.
- L'objet `ServletConfig` dispose de la méthode :
  `String getInitParameter(String name)`
  permettant de récupérer la valeur d'un paramètre d'initialisation du servlet.

Cycle de vie : Phase Service

- Méthode : `service()`
- Exécutée à chaque fois qu'une requête est reçue
- Suivant la nature de la requête, la méthode `service()` délègue le traitement à :
  - `doGet(HttpServletRequest request, HttpServletResponse response)`

    ou à
  - `doPost(HttpServletRequest request, HttpServletResponse response)`
- **`HttpServletRequest`** représente la requête reçue
- **`HttpServletResponse`** représente la réponse à envoyer au client

Cycle de vie : Phase Destruction

- Méthode `destroy()`
- Exécutée une seule fois à l'arrêt du servlet
- Utile pour libérer les ressources allouées

Structure de base du Servlet

- Un servlet doit étendre la classe abstraite `javax.servlet.http.HttpServlet`

- Sa structure de base (sous l'IDE Netbeans) est composée des méthodes :
  1. `processRequest(HttpServletRequest request, HttpServletResponse response)` qui traite la requête et génére la réponse.
  2. `doGet(HttpServletRequest request, HttpServletResponse response)` qui appelle la méthode `processRequest`
  3. `doPost(HttpServletRequest request, HttpServletResponse response)` qui appelle la méthode `processRequest`

Un exemple de servlet

```
package premierservlet;
import ...;

public class PremierServlet extends HttpServlet {
  protected void processRequest(HttpServletRequest request,
     HttpServletResponse response) throws ServletException, IOException {

      LocalTime heureActuelle = LocalTime.now();
      DateTimeFormatter dtf = DateTimeFormatter.ofPattern("hh:mm:ss");
      String heureActuelleFormat = heureActuelle.format(dtf);

    response.setContentType("text/html;charset=UTF-8");
    try(PrintWriter out = response.getWriter()) {
      out.println("<html>");
      out.println("<head>");
      out.println("<title>Premier servlet</title>");
      out.println("</head>");
      out.println("<body>");
      out.println("<h1>IL est " + heureActuelleFormat + "</h1>");
      out.println("</body>");
      out.println("</html>");
    }
  }
```

Un exemple de servlet (Suite)

```
    @Override
    protected void doGet(HttpServletRequest request,
      HttpServletResponse response) throws ServletException, IOException {

       processRequest(request, response);

    }

    @Override
    protected void doPost(HttpServletRequest request,
      HttpServletResponse response) throws ServletException, IOException {

       processRequest(request, response);

    }
}
```

Fichiers de configuration : context.xml

- Ce fichier permet de définir le contexte de l'application.
- Le contexte est une partie de l'URL d'accès de l'application.
- Le contexte permet d'identifier l'application web au sein du serveur web.

Exemple :

```
<?xml version="1.0" encoding="UTF-8"?>
<Context path="/PremierServlet"/>
```

Fichiers de configuration : web.xml

- Le fichier **`web.xml`** est le descripteur de déploiement de l'application web.
- Il peut contenir :
  - des paramètres d'initialisation de l'application web,
  - des déclarations de servlets et leur URL de mappage,
  - des parametres d'initialisation de servlet,
  - la définition de la durée de la session,
  - l'indication de la page d'accueil de l'application.

Exemple de fichier web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
   ...
   <servlet>
      <servlet-name>PremierServlet</servlet-name>
      <servlet-class>premierservlet.PremierServlet</servlet-class>
      <init-param>
         <param-name>nbLignes</param-name>
         <param-value>5</param-value>
      </init-param>
   </servlet>

   <servlet-mapping>
      <servlet-name>PremierServlet</servlet-name>
      <url-pattern>/PremierServlet</url-pattern>
   </servlet-mapping>

   <welcome-file-list>
      <welcome-file>accueil.html</welcome-file>
   </welcome-file-list>
   ...
</web-app>
```

Déclarer un servlet avec les annotations

- Le servlet peut également être déclaré à l'aide d'annotations.

Exemple :

```
@WebServlet(
        name = "PremierServlet",
        urlPatterns = {"/PremierServlet", "/index"},

        initParams = {
            @WebInitParam( name = "nbLignes", value = "5" ),
            @WebInitParam( name = "nbRow", value = "45" )
        }
)
public class PremierServlet extends HttpServlet {
    ...
}
```

URL d'accès à un servlet

- L'URL d'accès à un servlet est composée des éléments suivants :
  1. protocole http ;
  2. nom ou adresse ip du serveur
  3. numéro de port (présent uniquement lorsqu'il est différent de 80)
  4. contexte de l'application
  5. URL pattern du servlet

- Exemple :

```
http://localhost:8080/PremierServlet/index
```

Les Java Server Pages (JSP)

C'est quoi un JSP ?

- Un JSP est une page web (pas une classe Java) qui combine du HTML (pour la partie statique) et du code java (pour la partie dynamique).
- Le code Java est intégré dans le code HTML à l'aide de balises spéciales.
- Contrairement aux servlets, les JSP ne sont pas organisés suivant une logique de paquets, ils sont enregistrés dans les mêmes répertoires que les pages web.
- Dans le modèle MVC, les JSP sont utilisés pour la vue (la présentation de la réponse).

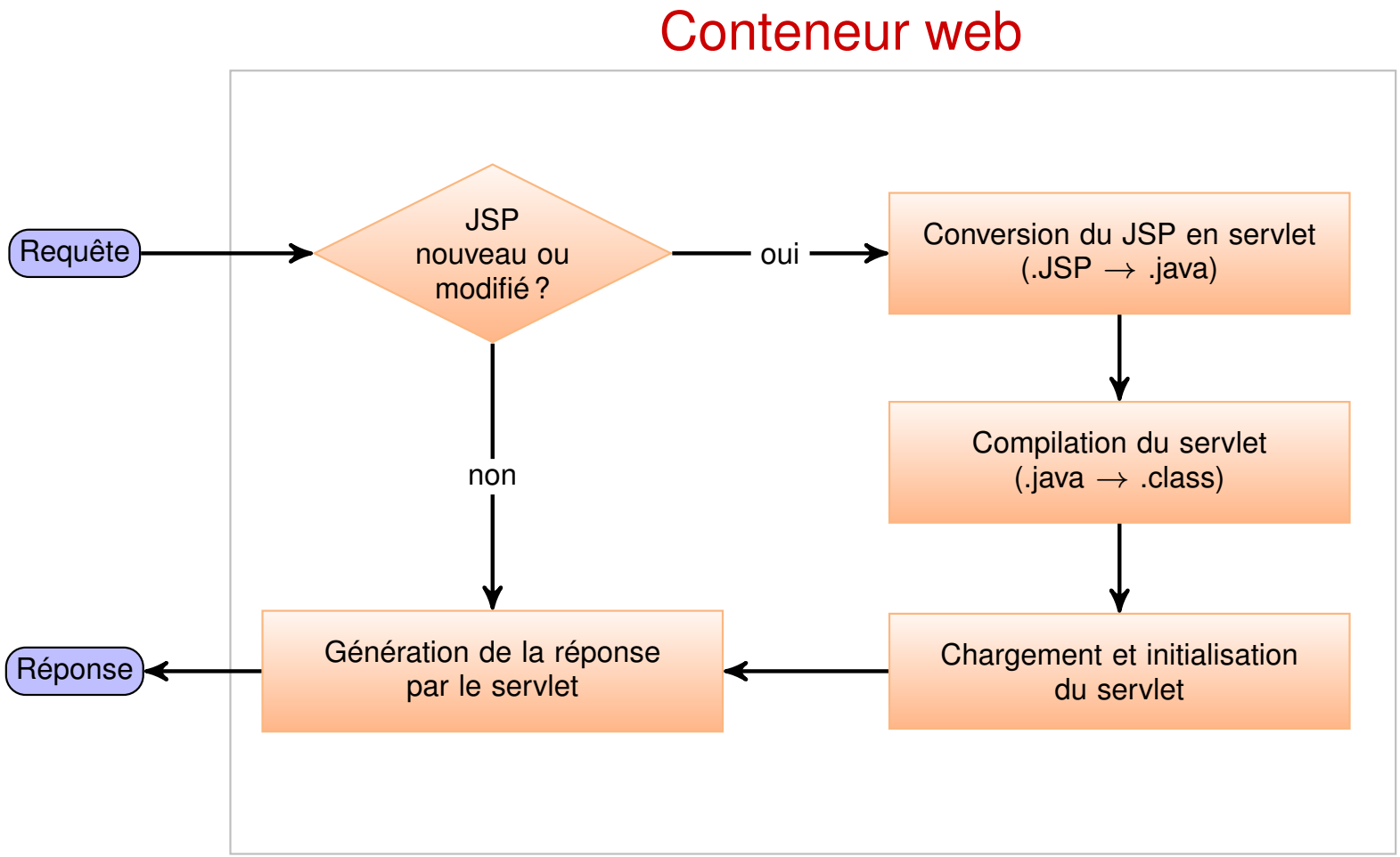
Exemple : Version JSP du servlet précédent

```
<%@page import="java.time.format.DateTimeFormatter"%>
<%@page import="java.time.LocalTime"%>
<%@page contentType="text/html" pageEncoding="UTF-8"%>

<%!
    LocalTime heureActuelle = LocalTime.now();
    DateTimeFormatter dtf = DateTimeFormatter.ofPattern("hh:mm:ss");
    String heureActuelleFormat = heureActuelle.format(dtf);
%>

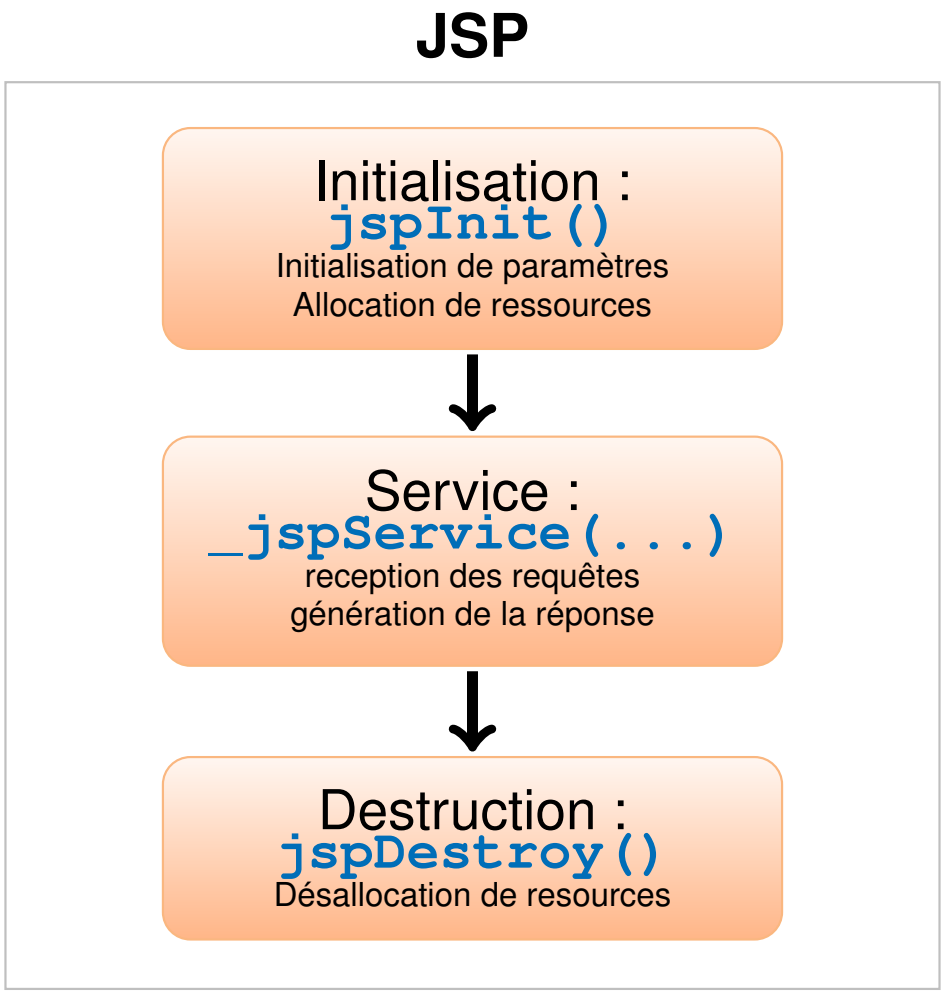
<!DOCTYPE html>
<html>
   <head>
      <meta http-equiv="Content-Type" content="text/html;
          charset=UTF-8">
      <title>Heure</title>
   </head>
   <body>
      <h1>Il est <%= heureActuelleFormat %></h1>
   </body>
</html>
```

Cycle de vie du JSP



Cycle de vie du JSP

Interface : `javax.servlet.jsp.HttpJspPage`



Contenu d'un JSP : contenu statique

- Le contenu statique peut être constitué de :
  - code HTML
  - CSS
  - Javascript
  - ...
- Lors de la conversion JSP $\rightarrow$ servlet le contenu statique reste inchangé.

Contenu d'un JSP : contenu dynamique

Le contenu dynamique est constitué des éléments suivants :

- Les directives
- Les éléments de scripts
- Les expressions EL
- Les éléments de propriétés `JSP:[set|get]`
- Les éléments `JSP:[include|forward]`
- L'élément `JSP:plugin`
- Les tags personnalisés

Les éléments de scripts

- Les éléments de scripts permettent d'insérer du code Java dans une page JSP.
- Il y a 4 types d'éléments de scripts :
  1. le commentaire pour insérer un commentaire JSP.
     Syntaxe : `<%-- commentaire --%>`
  2. la déclaration pour déclarer des variables et des méthodes.
     Syntaxe : `<%! déclaration %>`
  3. l'expression pour afficher la valeur d'une expression
     Syntaxe : `<%= expression %>`
  4. le scriptlet pour les blocs de traitements
     Syntaxe : `<% traitement %>`

Les déclarations

- Les déclarations sont utilisées pour déclarer les variables et les méthodes qui seront utilisées dans la page JSP.
- Syntaxe : `<%! déclaration %>`
- Exemple

```
<%!
    float prixUnitCahier = 100f;
    int quantite = 5;
    public float CalculerTotal(float prix, int quantite){
       float total = prix * quantite;
       return total;
    }
%>
```

- Remarque

  Il est possible d'insérer plusieurs balises déclaration dans une page JSP.

Les expressions

- Les expressions sont utilisées pour afficher la valeur d'une variable ou la valeur retournée par une méthode.
- L'expression est évaluée dynamiquement puis convertie en `String` avant son insertion dans la réponse.
- Syntaxe : `<%= expression %>`
- Exemple :

```
<body>
    Le prix d'un cahier est: <%= prixUnitCahier %><br/>
    Le nombre de cahier acheté est: <%= quantite %><br/>
    Le total à payer est:
      <%= CalculerTotal(prixUnitCahier, quantite) %>
</body>
```

Les scriptlets

- Les scriptlets sont utilisés pour insérer des blocs d'instructions (boucles, traitements conditionnels, etc.) dans la page JSP. Elles contiennent généralement les instructions de traitement de la requête.
- Syntaxe : `<% traitement %>`
- Exemple :

```
<body>
<%
    float aPayer = CalculerTotal(prixUnitCahier, quantite);
    if (aPayer >= 5000f) {
%>
       Vous avez une réduction de 1500 F.
<%  } else {%>
       <b>1500 F de réduction à partir de 5000 F d'achats.</b>
<%  }%>
</body>
```

Les scriptlets

- Exemple (autre écriture) :

```
<body>
<%
   float aPayer = CalculerTotal(prixUnitCahier, quantite);
   if (aPayer >= 500f) {
      out.println("Vous avez une réduction de 1500 F.");
   else {
      out.println("<b>1500 F de réduction à partir de 5000 F d'
          achats.</b>");
   }
%>
</body>
```

Les directives

- Les directives sont des instructions utiles au conteneur pour la traduction de la page JSP en servlet.
- Ces directives sont au nombre de 3 :
  - la directive `<%@ page %>`
  - la directive `<%@ include %>`
  - la directive `<%@ taglib %>`

La directive `<%@ page %>`

- La directive `<%@ page %>` permet de définir à l'aide de ses attributs un certain nombre de propriétés pour la page JSP .
- **Syntaxe**

```
<%@ page attribut="valeur" %>
```

- **Quelques attributs**
  - `contentType`
  - `import`
  - `errorPage`
  - `isErrorPage`
- Remarque : Plusieurs attributs peuvent être définis pour la directive `<%@ page %>`, il suffit de les séparer par des virgules.

L'attribut `contentType`

- L'attribut `contentType` définit le type de contenu de la réponse et optionnellement le jeu de caractères utilisé.
- **Exemple :**
  Le contenu de la réponse est du HTML

```
<%@ page contentType="text/html" %>
```

- Le contenu de la réponse est une image jpeg

```
<%@ page contentType="image/jpeg" %>
```

- Le contenu de la réponse est du HTML avec le jeu de caractères ISO-Latin1

```
<%@ page contentType="text/html:charset=ISO-8859-1" %>
```

L'attribut `import`

- L'attribut **`import`** permet de spécifier les paquets et/ou classes à importer comme le fait le mot clé **`import`**.
- **Exemple :**
  Importer la classe **`LocalTime`**

```
<%@page import="java.time.LocalTime"%>
```

- Importer toutes les classes du paquet **`java.util`**

```
<%@ page import="java.util.*" %>
```

- Importer la classe **`LocalTime`** et toutes les classes du paquet **`java.util`**

```
<%@ page import="java.time.LocalTime,java.util.*" %>
```

L'attribut `errorPage`

- L'attribut **`errorPage`** est utilisé pour spécifier l'adresse url de la page à afficher lorsqu'une erreur survient lors de l'exécution de la page JSP.

- **Exemple :**
  Afficher la page **`erreur.jsp`** lorsqu'une erreur survient lors de l'exécutions de cette page.

```
<%@ page errorPage="erreur.jsp" %>
```

L'attribut `isErrorPage`

- L'attribut **`isErrorPage`** permet d'indiquer si la page est une page d'erreur. Cet attribut vaut par défaut **`false`**.
- **Exemple :**
  Indiquer que la page est une page d'erreur.

```
<%@ page isErrorPage="true" %>
```

La directive <%@ include %>

- La directive <%@ include %> permet d'inclure le contenu d'un fragment de JSP dans une page JSP.
- L'inclusion est faite pendant la phase de conversion JSP → servlet.
- La directive <%@ include %> peut être insérée n'importe où dans la page JSP.
- **Syntaxe**

```
<%@ include file="url_relative" %>
```

- Remarque : <%@ include %> est généralement utilisée pour inclure un menu, un en-tête de page ou un bas de page.

La directive <%@ include %> : exemple

Exemple : `pagedate.jsp`

```
<%@page import="java.time.format.DateTimeFormatter"%>
<%@page import="java.time.LocalDate"%>

<%@page contentType="text/html" pageEncoding="UTF-8"%>

<!DOCTYPE html>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>Aujourd'hui</title>
    </head>
    <body>
        <%@include file="WEB-INF/jspf/dat.jspf" %>
    </body>
</html>
```

La directive <%@ include %>

Fragment dat.jspf

```
<%!
    LocalDate dateActuelle = LocalDate.now();
    DateTimeFormatter dtf = DateTimeFormatter.ofPattern("dd/MM/yyyy");
    String dateActuelleFormat = dateActuelle.format(dtf)
%>

<h1>Aujourd'hui nous sommes le <%= dateActuelleFormat %></h1>
```

Objets implicites

- Les objets implicites sont des variables prédéfinies et initialisées par le conteneur.
- Ils sont disponibles dans les expressions et les scriptlets.
- Quelques objets implicites :
  - out : représente l'objet permettant d'écrire sur la réponse (`PrintWriter`).
  - config : représente l'objet `ServletConfig` du servlet obtenu à partir de la page JSP.